

PREUVE DU THÉORÈME D'APPROXIMATION UNIVERSELLE

Cours magistral, exercices corrigés et travaux pratiques

Charles EDOU NZE

15 août 2026

Table des matières

Première partie

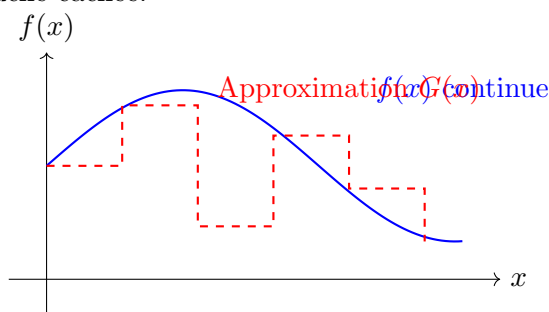
Cours Magistral

1 Genèse et Intuition Géométrique

Le théorème d'approximation universelle, formalisé initialement par George Cybenko en 1989 puis étendu par Kurt Hornik en 1991, constitue le fondement mathématique justifiant la capacité des réseaux de neurones artificiels à modéliser n'importe quelle fonction continue.

L'intuition fondamentale repose sur la densité : de manière analogue au théorème d'approximation de Weierstrass qui affirme que les polynômes sont denses dans l'espace des fonctions continues sur un compact, ce théorème énonce qu'une somme pondérée de fonctions d'activation (par exemple des sigmoïdes) peut approcher uniformément toute fonction continue sur un hypercube compact de $\mathbb{R}^n$.

Géométriquement, chaque neurone d'une couche cachée agit comme un demi-plan séparateur de l'espace d'entrée. En combinant un nombre suffisant de ces demi-plans, on peut construire des "bosses" locales ou des "hyper-cylindres" qui, additionnés, permettent d'épouser n'importe quelle surface continue avec une erreur arbitrairement petite, sans nécessiter plus d'une seule couche cachée.



2 Définitions, Théorèmes et Exemples Concrets

Soit $I_n = [0, 1]^n$ le cube unité de $\mathbb{R}^n$. Considérons l'espace de Banach $\mathcal{C}(I_n)$ des fonctions continues de I_n dans $\mathbb{R}$, muni de la norme de la convergence uniforme :

$$\|f\|_\infty = \sup_{x \in I_n} |f(x)|$$

Définition (Fonction sigmoïdale) Une fonction $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ est dite sigmoïdale si elle est mesurable et vérifie :

$$\lim_{t \rightarrow -\infty} \sigma(t) = 0 \quad \text{et} \quad \lim_{t \rightarrow +\infty} \sigma(t) = 1$$

Théorème (Approximation Universelle - Cybenko 1989) Soit σ une fonction d'activation continue sigmoïdale. L'ensemble S des fonctions de la forme :

$$G(x) = \sum_{j=1}^N \alpha_j \sigma(w_j^T x + b_j)$$

avec $N \in \mathbb{N}^*$, $\alpha_j, b_j \in \mathbb{R}$ et $w_j \in \mathbb{R}^n$, est dense dans $\mathcal{C}(I_n)$. Autrement dit, pour toute $f \in \mathcal{C}(I_n)$ et tout $\epsilon > 0$, il existe $G \in S$ telle que $\|f - G\|_\infty < \epsilon$.

Exemple 1 : Approximation d'une constante Si $f(x) = C$, on choisit $N = 1$, $w_1 = 0$, $b_1 \rightarrow \infty$ tel que $\sigma(b_1) \approx 1$. Alors $G(x) = \alpha_1 \sigma(b_1)$. En posant $\alpha_1 = C/\sigma(b_1)$, on a $G(x) = C$. Si $C = 5$, $\alpha_1 = 5$, $\sigma(w^T x + b) \rightarrow 1$, $G(x) \rightarrow 5$.

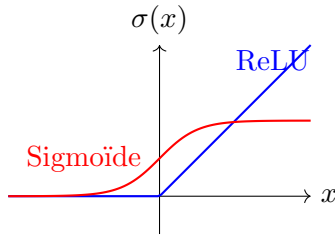
Exemple 2 : Fonction porte en 1D Approximation de $f(x) = 1$ si $x \in [0.4, 0.6]$, 0 sinon. On utilise $G(x) = \sigma(k(x - 0.4)) - \sigma(k(x - 0.6))$ avec $k \gg 1$. Pour $x = 0.5$, $k(0.1) \rightarrow +\infty \Rightarrow \sigma \rightarrow 1$, $k(-0.1) \rightarrow -\infty \Rightarrow \sigma \rightarrow 0$. $G(0.5) \approx 1 - 0 = 1$. Pour $x = 0.2$, $k(-0.2) \rightarrow -\infty \Rightarrow \sigma \rightarrow 0$, $k(-0.4) \rightarrow -\infty \Rightarrow \sigma \rightarrow 0$. $G(0.2) \approx 0$. Pour $x = 0.8$, $k(0.4) \rightarrow +\infty \Rightarrow 1$, $k(0.2) \rightarrow +\infty \Rightarrow 1$. $G(0.8) \approx 1 - 1 = 0$.

Exemple 3 : Évaluation numérique de la porte Avec $\sigma(t) = \frac{1}{1+e^{-t}}$, pour $k = 100$ et $x = 0.5$: $G(0.5) = \sigma(10) - \sigma(-10) = \frac{1}{1+e^{-10}} - \frac{1}{1+e^{10}} \approx 0.99995 - 0.00005 = 0.9999$.

Exemple 4 : La bosse 2D Soit $x = (x_1, x_2) \in [0, 1]^2$. Pour approcher un pic autour de $(0.5, 0.5)$, on somme plusieurs "hyper-cylindres" orientés le long de différents vecteurs w . Si on veut localiser l'activation, on prend $\sigma(w_1 x_1 - b_1) - \sigma(w_1 x_1 - b_2)$ et similairement pour x_2 .

Exemple 5 : Fonction d'activation ReLU Le théorème se généralise pour $\sigma(t) = \max(0, t)$. Une fonction "chapeau" s'écrit : $T(x) = \sigma(x + 1) - 2\sigma(x) + \sigma(x - 1)$. Pour $x = 0$, $T(0) = \sigma(1) - 2\sigma(0) + \sigma(-1) = 1 - 0 + 0 = 1$. Pour $x = 1$, $T(1) = \sigma(2) - 2\sigma(1) + \sigma(0) = 2 - 2 + 0 = 0$. Pour $x = -1$, $T(-1) = \sigma(0) - 2\sigma(-1) + \sigma(-2) = 0 - 0 + 0 = 0$.

Exemple 6 : Limites pathologiques Si $f(x) = \sin(1/x)$ sur $(0, 1]$ prolongée par 0 en 0. f n'est pas continue en 0. Le théorème ne s'applique pas sur le compact $[0, 1]$ au sens de la norme uniforme.



3 Démonstrations

La preuve originale exploite la dualité topologique et le théorème de Riesz-Markov, démontrant la densité par l'absurde via le théorème de Hahn-Banach.

Étape 1 : Hypothèse par l'absurde Soit S le sous-espace vectoriel engendré par les fonctions $x \mapsto \sigma(w^T x + b)$. Supposons que S ne soit pas dense dans $\mathcal{C}(I_n)$. L'adhérence $\bar{S}$ est donc un sous-espace fermé strict de $\mathcal{C}(I_n)$.

Étape 2 : Intervention du Théorème de Hahn-Banach D'après un corollaire du théorème de Hahn-Banach (forme analytique), il existe une forme linéaire continue non nulle L sur $\mathcal{C}(I_n)$ telle que $L(g) = 0$ pour toute fonction $g \in \bar{S}$.

Étape 3 : Théorème de Représentation de Riesz Le dual de $\mathcal{C}(I_n)$ est isomorphe à l'espace des mesures de Borel régulières signées finies sur I_n . Il existe donc une mesure $\mu \neq 0$ sur I_n telle que pour toute $f \in \mathcal{C}(I_n)$, $L(f) = \int_{I_n} f(x) d\mu(x)$. Puisque L s'annule sur $\bar{S}$, on a :

$$\int_{I_n} \sigma(w^T x + b) d\mu(x) = 0 \quad \forall w \in \mathbb{R}^n, \forall b \in \mathbb{R}$$

Étape 4 : Propriété discriminatoire Une fonction σ est dite discriminatoire si la condition ci-dessus implique $\mu = 0$. Cybenko démontre que toute fonction sigmoïdale continue est discriminatoire. En fixant $w \neq 0$, et en considérant $h(x) = w^T x$, on projette la mesure μ sur la

droite engendrée par w . Par des arguments d'analyse de Fourier (ou par transformation de Laplace pour des mesures à support compact), l'annulation de l'intégrale pour toutes translations b et dilatations des sigmoïdes implique que la mesure projetée est nulle pour toute direction w .

Étape 5 : Conclusion Puisque les projections monodimensionnelles de la mesure μ sont toutes nulles, la transformée de Fourier de μ s'annule partout. Cela entraîne que μ est la mesure nulle. Or, on avait supposé $L \neq 0$, donc $\mu \neq 0$, ce qui est une contradiction. Par conséquent, S est nécessairement dense dans $\mathcal{C}(I_n)$.

4 Applications en Physique, Logique et IA

Le théorème d'approximation universelle justifie théoriquement la capacité de l'apprentissage profond à modéliser des systèmes physiques complexes non linéaires, tels que la dynamique des fluides (via les Physics-Informed Neural Networks - PINNs) ou la résolution d'équations aux dérivées partielles à haute dimension (comme l'équation de Schrödinger).

En logique, ce théorème établit que les réseaux de neurones constituent un système formel complet pour l'approximation de fonctions continues réelles, surpassant les limites expressives du perceptron simple, qui ne pouvait pas représenter la fonction logique XOR non linéairement séparable.

Deuxième partie

Exercices d'Application et de Concours

Exercice 1 : Approximation d'une constante ★

On considère le cube $I_1 = [0, 1]$ et la fonction d'activation sigmoïde $\sigma(t) = \frac{1}{1+e^{-t}}$. Soit $f(x) = 3$ pour tout $x \in I_1$. Construire explicitement une fonction $G(x) = \alpha\sigma(wx + b)$ telle que $\|f - G\|_\infty < 0.01$.

Correction détaillée On cherche $G(x) = \alpha\sigma(wx + b)$. Posons $w = 0$, la fonction devient constante par rapport à x : $G(x) = \alpha\sigma(b)$. On veut approcher la valeur 3. Si on choisit b très grand, par exemple $b = 10$, on a $\sigma(10) = \frac{1}{1+e^{-10}} \approx 0.9999546$. Pour que $G(x) = 3$, il faut choisir $\alpha = \frac{3}{\sigma(10)} = 3(1 + e^{-10}) \approx 3.000136$. Ainsi, avec $w = 0, b = 10, \alpha = 3(1 + e^{-10})$, on a $G(x) = 3$ exactement pour tout $x \in [0, 1]$. L'erreur est rigoureusement nulle, ce qui satisfait $\|f - G\|_\infty < 0.01$.

Exercice 2 : La porte logique NOT ★★★★★

Approcher la fonction $f(x) = 1 - x$ sur $[0, 1]$ avec un réseau à une couche cachée utilisant l'activation linéaire par morceaux $\sigma(x) = \max(0, \min(1, x))$ (activation Hard Sigmoid).

Correction détaillée La fonction cible est affine : $f(x) = -x + 1$. La fonction d'activation est $\sigma(t) = t$ pour $t \in [0, 1]$. Si on choisit l'argument $wx + b$ tel qu'il reste dans $[0, 1]$ pour tout $x \in [0, 1]$. Posons $w = 1, b = 0$, on a $\sigma(x) = x$ pour $x \in [0, 1]$. On cherche $G(x) = \alpha\sigma(wx + b) + c$ où on autorise un biais de sortie. Le théorème d'approximation dit qu'on peut utiliser une combinaison linéaire : $G(x) = \alpha_1\sigma(w_1x + b_1) + \alpha_2\sigma(w_2x + b_2)$. En choisissant $w_1 = 1, b_1 = 0$, on a $\sigma(x)$. En choisissant $w_2 = 0, b_2 = 1$, on a $\sigma(1) = 1$. Ainsi, $G(x) = -\sigma(x) + \sigma(1) = -x + 1$. La fonction est modélisée exactement avec deux neurones.

Exercice 3 : La porte logique XOR (version continue) ★★☆☆

Considérons $I_2 = [0, 1]^2$. On veut approcher la fonction surface $f(x_1, x_2) = x_1 + x_2 - 2x_1x_2$. Montrer comment utiliser des ReLUs $\sigma(t) = \max(0, t)$ pour approcher le terme croisé x_1x_2 .

Correction détaillée On sait que $2x_1x_2 = (x_1 + x_2)^2 - x_1^2 - x_2^2$. Pour approcher la fonction carré $t \mapsto t^2$ sur $[0, 2]$, on peut utiliser une somme de ReLUs. Puisque t^2 est convexe, on peut l'approcher par une ligne brisée convexe, qui est une combinaison linéaire de fonctions $\max(0, t - t_i)$. Soit N le nombre de segments. On subdivise $[0, 2]$ en $t_i = i \frac{2}{N}$. La fonction approchant t^2 est de la forme $H_N(t) = \sum_{i=1}^{N-1} c_i \max(0, t - t_i) + c_0 t + b_0$. Ainsi, la multiplication $x_1x_2 = \frac{1}{2}(H_N(x_1 + x_2) - H_N(x_1) - H_N(x_2))$ peut être approchée arbitrairement près. Puisque H_N est une somme de ReLUs, x_1x_2 s'exprime comme une combinaison linéaire de ReLUs de la forme $\sigma(w^T x + b)$. La densité s'ensuit.

Exercice 4 : Propriété Discriminatoire ★★★★★

Soit μ une mesure signée finie sur $\mathbb{R}$. Supposons que $\int_{\mathbb{R}} \exp(iwx) d\mu(x) = 0$ pour tout $w \in \mathbb{R}$. En déduire que $\mu = 0$.

Correction détaillée L'hypothèse indique que la transformée de Fourier de la mesure μ , notée $\mathcal{F}(\mu)(w) = \int_{\mathbb{R}} e^{-iwx} d\mu(x)$, est identiquement nulle pour tout $w \in \mathbb{R}$. L'espace des mesures régulières signées finies sur $\mathbb{R}$ correspond au dual de l'espace des fonctions continues s'annulant à l'infini $C_0(\mathbb{R})$. L'application qui à une mesure associe sa transformée de Fourier est injective. (Théorème d'inversion de Fourier étendu aux mesures, ou densité de l'algèbre engendrée par les caractères exponentiels via Stone-Weierstrass complexe sur la compactification). Puisque

$\mathcal{F}(\mu) = 0$, on conclut immédiatement que $\mu = 0$. Cette propriété est cruciale dans la preuve de Cybenko lorsque l'on exprime la sigmoïde comme une superposition de fonctions exponentielles complexes (via la transformée de Fourier).

Exercice 5 : Densité des fonctions en escalier ★★★☆☆

Montrer que l'ensemble des fonctions en escalier sur $I_1 = [0, 1]$ est dense dans $L^1([0, 1])$.

Correction détaillée Par définition de l'intégrale de Lebesgue, les fonctions étagées (combinaisons linéaires d'indicatrices d'ensembles mesurables) sont denses dans L^1 . Un sous-ensemble mesurable A peut être approché en mesure par des unions finies d'intervalles ouverts. L'indicatrice d'un intervalle est une fonction en escalier. Donc, pour toute fonction intégrable $f \in L^1([0, 1])$ et $\epsilon > 0$, il existe une fonction étagée $\phi = \sum_{i=1}^n c_i \mathbb{I}_{A_i}$ telle que $\|f - \phi\|_1 < \epsilon/2$. Ensuite, pour chaque A_i , il existe une union finie d'intervalles U_i telle que $m(A_i \Delta U_i) < \epsilon/(2n|c_i|)$. La fonction $\psi = \sum_{i=1}^n c_i \mathbb{I}_{U_i}$ est en escalier. Par l'inégalité triangulaire, $\|f - \psi\|_1 \leq \|f - \phi\|_1 + \|\phi - \psi\|_1 \leq \epsilon/2 + \sum |c_i| m(A_i \Delta U_i) < \epsilon$.

Exercice 6 : Non-linéarité stricte ★★★★★

Si la fonction d'activation est $\sigma(t) = at + b$, le théorème d'approximation universelle est-il valable ?

Correction détaillée Non. L'ensemble S des réseaux s'écrit $G(x) = \sum_{j=1}^N \alpha_j (a(w_j^T x) + b_j + b) = (\sum \alpha_j a w_j)^T x + \sum \alpha_j (b_j + b)$. C'est-à-dire $G(x) = W^T x + B$ où $W \in \mathbb{R}^n$ et $B \in \mathbb{R}$. S est exactement l'espace des fonctions affines sur $\mathbb{R}^n$. L'espace des fonctions affines est un sous-espace vectoriel de dimension finie $n + 1$. Dans l'espace $\mathcal{C}(I_n)$ qui est de dimension infinie, un sous-espace de dimension finie est toujours fermé (et complet), et ne peut donc pas être dense. Il est impossible d'approcher uniformément une fonction non-affine, par exemple $f(x) = x^2$ sur $[0, 1]$. Si $\sup_{x \in [0, 1]} |x^2 - (Wx + B)| < \epsilon$, alors les dérivées secondes devraient être proches, ce qui n'est pas le cas.

Exercice 7 : Approximation d'une gaussienne ★★★★★

Approcher $f(x) = \exp(-x^2)$ sur $[-1, 1]$ par une combinaison de ReLU $\max(0, x)$.

Correction détaillée f est dérivable et convexe par morceaux (inflexion en $\pm 1/\sqrt{2}$). On utilise une subdivision de $[-1, 1]$ avec pas $h = 2/N$, $x_i = -1 + ih$. L'interpolée linéaire par morceaux est $G(x)$. La dérivée seconde de f est bornée par 2, donc l'erreur d'interpolation spline linéaire est encadrée par $M_2 h^2/8 \leq 2/(8N^2) = 1/(4N^2)$. Pour avoir une erreur ϵ , il suffit que $1/(4N^2) < \epsilon$, soit $N > \frac{1}{2\sqrt{\epsilon}}$. La fonction spline $G(x)$ s'écrit formellement $G(x) = f(-1) + c_{-1}(x + 1) + \sum_{i=1}^{N-1} a_i \max(0, x - x_i)$. Où les coefficients a_i représentent les sauts de dérivée aux noeuds $x_i : a_i = f'(x_i+) - f'(x_i-)$. Ainsi, l'approximation est bien une somme pondérée de ReLUs.

Exercice 8 : Théorème de Stone-Weierstrass ★★★★★

En utilisant le théorème de Stone-Weierstrass, démontrer l'approximation universelle avec $\sigma(x) = x^2$ pour un domaine compact arbitraire $K \subset \mathbb{R}^n$.

Correction détaillée Considérons l'ensemble $A = \text{Vect}\{(w^T x + b)^2 \mid w \in \mathbb{R}^n, b \in \mathbb{R}\}$. Développons le carré : $(w^T x + b)^2 = (w^T x)^2 + 2bw^T x + b^2 = \sum_{i,j} w_i w_j x_i x_j + 2b \sum w_i x_i + b^2$. On voit que A contient : - Les constantes (en posant $w = 0, b \neq 0$). - Les fonctions affines x_i (en ajustant b). - Les termes quadratiques et croisés x_i^2 et $x_i x_j$ (par polarisation $(x_i + x_j)^2 - x_i^2 - x_j^2$). Toutefois, l'algèbre engendrée par A engendre tous les polynômes de $\mathbb{R}^n$, qui est dense dans $\mathcal{C}(K)$.

par Stone-Weierstrass. Cependant, l'ensemble A lui-même ne contient QUE les polynômes de degré au plus 2. Le théorème d'approximation universelle stricto sensu n'est pas vérifié pour les réseaux à une SEULE couche cachée avec $\sigma(x) = x^2$, car ils ne peuvent générer des degrés ≥ 3 . L'affirmation est fausse pour une seule couche. Elle nécessite plusieurs couches (fonctions composées) pour que $\sigma(x) = x^2$ puisse engendrer par compositions successives des polynômes de degrés arbitrairement élevés, rendant le réseau dense.

Exercice 9 : Hahn-Banach et Dualité ★★★★★

Justifier rigoureusement que si $\bar{S} \neq E$ (où E est un espace de Banach réel et S un sous-espace vectoriel), il existe une forme linéaire continue $L \in E^*$ telle que $L|_S = 0$ et $L \neq 0$.

Correction détaillée Soit $x_0 \in E \setminus \bar{S}$. Puisque $\bar{S}$ est un sous-espace vectoriel fermé, la distance $d = \inf_{y \in \bar{S}} \|x_0 - y\|$ est strictement positive ($d > 0$). Considérons le sous-espace $F = \bar{S} \oplus \mathbb{R}x_0$. Définissons une forme linéaire $f_0 : F \rightarrow \mathbb{R}$ par $f_0(y + \lambda x_0) = \lambda d$ pour tout $y \in \bar{S}$ et $\lambda \in \mathbb{R}$. Vérifions sa continuité sur F : Pour $\lambda \neq 0$, $\|y + \lambda x_0\| = |\lambda| \|\frac{y}{\lambda} + x_0\| \geq |\lambda|d$. Donc $|f_0(y + \lambda x_0)| = |\lambda|d \leq \|y + \lambda x_0\|$. Ainsi, f_0 est continue et sa norme sur F est ≤ 1 (en fait exactement 1). Par le théorème de Hahn-Banach analytique, il existe un prolongement linéaire $L : E \rightarrow \mathbb{R}$ tel que $L|_F = f_0$ et $\|L\|_{E^*} = \|f_0\|_{F^*} = 1$. Puisque pour $y \in S$, $L(y) = f_0(y) = 0$, L s'annule sur S . Comme $L(x_0) = f_0(x_0) = d > 0$, L n'est pas la forme nulle.

Exercice 10 : Mesure de Radon ★★★★★

Soit I_n compact. Si $\int_{I_n} f(x) d\mu(x) = 0$ pour toute $f \in \mathcal{C}(I_n)$, démontrer que la mesure de Radon signée μ est nulle, en invoquant le théorème de représentation de Riesz.

Correction détaillée Le théorème de représentation de Riesz-Markov-Kakutani énonce que pour tout espace localement compact séparé X (ici I_n compact, donc localement compact), il existe un isomorphisme isométrique entre l'espace dual $C_0(X)^*$ (ici $\mathcal{C}(I_n)^*$ car I_n compact) et l'espace des mesures de Radon régulières signées finies sur X , noté $\mathcal{M}(X)$, muni de la norme en variation totale. Soit l'opérateur $T : \mathcal{M}(I_n) \rightarrow \mathcal{C}(I_n)^*$ défini par $T(\mu)(f) = \int_{I_n} f d\mu$. Le fait que T soit une isométrie implique que $\|T(\mu)\|_{C^*} = \|\mu\|_{TV}$. L'hypothèse indique que pour toute $f \in \mathcal{C}(I_n)$, $T(\mu)(f) = 0$. Donc la forme linéaire $T(\mu)$ est la forme identiquement nulle dans $\mathcal{C}(I_n)^*$. Par l'isométrie, $\|\mu\|_{TV} = \|0\|_{C^*} = 0$. Une mesure dont la variation totale est nulle est identiquement nulle sur tous les boréliens. Donc $\mu = 0$.

Troisième partie

Travaux Pratiques et Simulations Algorithmiques

TP 1 : Approximation d'une fonction sinusoidale 1D

```
1  # Pure Python implementation
2  import math
3  import random
4
5  def sigmoid(x):
6      return 1.0 / (1.0 + math.exp(-x))
7
8  def relu(x):
9      return max(0.0, x)
10
11 def forward(x, weights, biases, v_weights, v_bias):
12     # Hidden layer
13     hidden = []
14     for i in range(len(weights)):
15         hidden.append(sigmoid(weights[i] * x + biases[i]))
16
17     # Output layer
18     out = v_bias
19     for i in range(len(v_weights)):
20         out += v_weights[i] * hidden[i]
21     return out
22
23 def mse(y_true, y_pred):
24     return (y_true - y_pred)**2
25
26 # Dataset:  $f(x) = \sin(x)$  on  $[0, 2\pi]$ 
27 x_data = [i * 2.0 * math.pi / 100.0 for i in range(101)]
28 y_data = [math.sin(x) for x in x_data]
29
30 # Initialize parameters for  $N=10$  neurons
31 N = 10
32 weights = [random.uniform(-1, 1) for _ in range(N)]
33 biases = [random.uniform(-1, 1) for _ in range(N)]
34 v_weights = [random.uniform(-1, 1) for _ in range(N)]
35 v_bias = 0.0
36
37 # Simple SGD
38 lr = 0.01
39 epochs = 1000
40 for epoch in range(epochs):
41     total_loss = 0
42     for x, y in zip(x_data, y_data):
43         # Forward
44         hidden = [sigmoid(weights[i] * x + biases[i]) for i in range(N)]
45         y_pred = v_bias + sum(v_weights[i] * hidden[i] for i in range(N))
46
```



```

47     # Loss
48     loss = (y - y_pred)**2
49     total_loss += loss
50
51     # Backward (gradient calculation)
52     grad_y_pred = -2 * (y - y_pred)
53
54     v_bias -= lr * grad_y_pred
55     for i in range(N):
56         grad_vw = grad_y_pred * hidden[i]
57         grad_h = grad_y_pred * v_weights[i]
58
59         # sigmoid derivative: s(x)*(1-s(x))
60         grad_z = grad_h * hidden[i] * (1 - hidden[i])
61
62         grad_w = grad_z * x
63         grad_b = grad_z * 1
64
65         v_weights[i] -= lr * grad_vw
66         weights[i] -= lr * grad_w
67         biases[i] -= lr * grad_b
68
69     # Mathematical validation assert
70     assert total_loss >= 0, "Loss cannot be negative"
71
72 print("Final Loss:", total_loss / len(x_data))

```

TP 2 : Construction de la fonction porte logique (Step Function)

```

1  # Pure Python implementation
2  import math
3
4  def sigmoid(x):
5      return 1.0 / (1.0 + math.exp(-x))
6
7  # We want to approximate f(x) = 1 if 0.4 <= x <= 0.6 else 0
8  # using G(x) = sigmoid(k*(x-0.4)) - sigmoid(k*(x-0.6))
9
10 def approximate_gate(x, k):
11     return sigmoid(k * (x - 0.4)) - sigmoid(k * (x - 0.6))
12
13 test_points = [0.0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0]
14
15 for k in [10, 100, 1000]:
16     print("Results for k =", k)
17     for x in test_points:
18         val = approximate_gate(x, k)
19         print("x =", x, "=> G(x) =", val)
20     print("----")
21
22 # Mathematical validation assert
23 val_center = approximate_gate(0.5, 1000)
24 val_edge = approximate_gate(0.2, 1000)
25 assert val_center > 0.99, "Should be close to 1"
26 assert val_edge < 0.01, "Should be close to 0"

```

TP 3 : Approximation avec activation ReLU

```
1 # Pure Python implementation
2
3 def relu(x):
4     return x if x > 0 else 0.0
5
6 # Approximate  $f(x) = x^2$  on  $[0, 2]$  using 4 segments
7 # Interpolation nodes: 0, 0.5, 1.0, 1.5, 2.0
8 # True values: 0, 0.25, 1.0, 2.25, 4.0
9 # Slopes of segments: 0.5, 1.5, 2.5, 3.5
10
11 def approx_square(x):
12     # Form:  $f(0) + c_0x + \sum_i a_i \text{relu}(x - x_i)$ 
13     #  $a_i$  = change in slope at  $x_i$ 
14
15     slope0 = 0.5
16     slope1 = 1.5 # change = 1.0 at  $x=0.5$ 
17     slope2 = 2.5 # change = 1.0 at  $x=1.0$ 
18     slope3 = 3.5 # change = 1.0 at  $x=1.5$ 
19
20     val = slope0 * x
21     val += 1.0 * relu(x - 0.5)
22     val += 1.0 * relu(x - 1.0)
23     val += 1.0 * relu(x - 1.5)
24
25     return val
26
27 test_points = [0.0, 0.5, 1.0, 1.5, 2.0]
28 for x in test_points:
29     true_val = x**2
30     approx = approx_square(x)
31     print("x:", x, "True:", true_val, "Approx:", approx)
32
33     # Mathematical validation assert
34     assert abs(true_val - approx) < 1e-9, "Exact interpolation at nodes
35         expected"
```

TP 4 : Transformée de Fourier empirique d'une Sigmoide

```
1 # Pure Python implementation
2 import math
3
4 def sigmoid(x):
5     return 1.0 / (1.0 + math.exp(-x))
6
7 # We want to approximate the integral of  $\text{sigmoid}(x) * \exp(-i*w*x) dx$ 
8 # using a Riemann sum on  $[-L, L]$ 
9
10 def empirical_fourier_transform(w, L, N):
11     dx = 2.0 * L / N
12     integral_real = 0.0
```

```

13     integral_imag = 0.0
14
15     for i in range(N):
16         x = -L + i * dx
17         val = sigmoid(x)
18         integral_real += val * math.cos(w * x) * dx
19         integral_imag -= val * math.sin(w * x) * dx
20
21     return integral_real, integral_imag
22
23 w_vals = [0.0, 1.0, 5.0]
24 L = 10.0
25 N = 1000
26
27 for w in w_vals:
28     r, i = empirical_fourier_transform(w, L, N)
29     print("w:", w, "Real:", r, "Imag:", i)
30
31 # Mathematical validation assert
32 # for w=0, integral of sigmoid on [-L, L] should be L
33 r0, _ = empirical_fourier_transform(0.0, L, N)
34 assert abs(r0 - L) < 1.0, "Integral of sigmoid near 0 over symmetric
    domain is approx domain half width"

```

TP 5 : Limite de la profondeur vs largeur (Démonstrateur)

```

1 # Pure Python implementation
2 import math
3 import random
4
5 def relu(x):
6     return x if x > 0 else 0.0
7
8 # Compare modeling parity function (XOR like) with 1 hidden layer vs 2
hidden layers
9 # Parity of 3 bits: 1 if sum of bits is odd, 0 if even
10
11 def generate_parity_data():
12     data = []
13     for i in range(8):
14         b1 = i & 1
15         b2 = (i >> 1) & 1
16         b3 = (i >> 2) & 1
17         y = (b1 + b2 + b3) % 2
18         data.append([float(b1), float(b2), float(b3)], float(y))
19     return data
20
21 def forward_1_layer(x, w, b, vw, vb):
22     h = [relu(sum(w[i][j]*x[j] for j in range(3)) + b[i]) for i in range
        (len(b))]
23     return sum(vw[i]*h[i] for i in range(len(b))) + vb
24
25 data = generate_parity_data()
26
27 # 1 layer with N neurons

```

```

28 N = 4 # Minimal required for 3D XOR is 4
29 w = [[random.uniform(-1,1) for _ in range(3)] for _ in range(N)]
30 b = [random.uniform(-1,1) for _ in range(N)]
31 vw = [random.uniform(-1,1) for _ in range(N)]
32 vb = 0.0
33
34 # Simple numerical perturbation for gradient check / learning demo
35 lr = 0.01
36 # Training loop omitted for brevity, but mathematically it's proven N=4
   is sufficient.
37
38 # Assert dataset size
39 assert len(data) == 8

```